

SYNTHETIX · SECURITY REVIEW

Synthetix SNX Vaults.

A security review of SNX Vaults.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Synthetix	SNX Vaults	Guardian	March 19 – April 25, 2025	May 2, 2025

§ 00

Contents.

72 pages — overview, confidence ranking, scope, 49 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 34 findings	P. 11
	Remediation Review 1 · 10 findings	P. 54
	Remediation Review 2 · 5 findings	P. 66
§ 05	Disclaimer	P. 71
§ 06	About Guardian	P. 72

§ 01

Overview.

Guardian reviewed SNX Vaults across 3 review rounds. The final report includes 2 Critical, 5 High, 13 Medium, 29 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	Synthetixio/vaults-contracts
CLIENT	METHODOLOGY
Synthetix	Smart contract · Manual · Main Review · Remediation Review 1 · Remediation Review 2
PROJECT	MAIN REVIEW COMMIT
SNX Vaults	29b8169a8758
PREPARED BY	REMEDATION REVIEW 1 COMMIT
Guardian	9fbb46787407
AUDIT WINDOW	REMEDATION REVIEW 2 COMMIT
March 19 – April 25, 2025	38d5e0e1b609
FINAL REPORT	
May 2, 2025	

1.1 Confidence Ranking

2 / 5

Low Confidence

Guardian assigns a Confidence Ranking of 2 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	2	0	0	0	2
■ High	5	0	2	0	3
■ Medium	13	0	7	0	6
■ Low	29	0	15	0	14
■ Informational	0	0	0	0	0
All severities	49	0	24	0	25

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence ASSIGNED	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed SNX Vaults in `Synthetixio/vaults-contracts`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Unrestricted Flash Loan Callback	Validation	CRITICAL	RESOLVED	p. 11
C-02	Mismatch in the redemption mechanism	Logical Error	CRITICAL	RESOLVED	p. 12
H-01	Deposits Might Change Funding Rate Direction	Warning	HIGH	ACKNOWLEDGED	p. 14
H-02	_performanceFeeHighWaterMark Is Incorrectly Updated	Logical Error	HIGH	RESOLVED	p. 15
H-03	Realized PNL Breaks Delta Neutrality	Logical Error	HIGH	RESOLVED	p. 16
H-04	Fee-on-fee Calculation Inflates Vault Fees	Logical Error	HIGH	RESOLVED	p. 17
M-01	Potential Underflow in totalAssets Function	Logical Error	MEDIUM	RESOLVED	p. 19
M-02	Excessive Swap Amounts Risk Failing	Warning	MEDIUM	ACKNOWLEDGED	p. 20
M-03	Repeated Tiny Deposits	Warning	MEDIUM	RESOLVED	p. 21
M-04	Missing Proper Slippage Checks	Code Best Practices	MEDIUM	RESOLVED	p. 22
M-05	Redeem Fail When Very Low Total Deposits	Logical Error	MEDIUM	ACKNOWLEDGED	p. 23
M-06	Potential DoS via Unexpected ETH	Logical Error	MEDIUM	RESOLVED	p. 24
M-07	Leftover SNX_USD are stuck in the contrat	Warning	MEDIUM	ACKNOWLEDGED	p. 25

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-01	Unnecessary Zero-Fee Transfers	Gas Optimization	LOW	RESOLVED	p. 26
L-02	Excessive Gas Usage in Deposit Flow	Warning	LOW	ACKNOWLEDGED	p. 27
L-03	Lack Of ERC4626 View Functions	Warning	LOW	ACKNOWLEDGED	p. 28
L-04	Lack Of a Double Step Transfer Ownership Pattern	Code Best Practices	LOW	ACKNOWLEDGED	p. 29
L-05	Deposits May Revert	Validation	LOW	RESOLVED	p. 32
L-06	Last User Can Not Fully Redeem	Logical Error	LOW	ACKNOWLEDGED	p. 33
L-07	Flashloan Fee Is Not Accounted	Warning	LOW	ACKNOWLEDGED	p. 34
L-08	Unset Referrer Field	Warning	LOW	RESOLVED	p. 35
L-09	Vault Is More Exposed To Liquidation	Warning	LOW	RESOLVED	p. 36
L-10	Admin Should Call rebalance Frequently	Warning	LOW	ACKNOWLEDGED	p. 37
L-11	Theoretical First-Depositor Inflation Attack	Warning	LOW	ACKNOWLEDGED	p. 38
L-12	Order Cancellation Exposes Vault To Market	Warning	LOW	ACKNOWLEDGED	p. 39
L-13	Missing Max. Open Interest Checks	Validation	LOW	ACKNOWLEDGED	p. 40
L-14	Debt Can Be Artificially Inflated	Warning	LOW	RESOLVED	p. 42
L-15	Ineffective Swap Deadline	Validation	LOW	ACKNOWLEDGED	p. 43
L-16	Redemption Failure Due To Aave State	Warning	LOW	ACKNOWLEDGED	p. 45
L-17	Precision Loss in Redemption Calculation	Logical Error	LOW	RESOLVED	p. 47

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-18	Cache Total Fee Amount	Gas Optimization	LOW	ACKNOWLEDGED	p. 49
L-19	maxRedeem Function Ignores maxRedemptionPercent	Warning	LOW	RESOLVED	p. 50
L-20	Missing Fee Setting Constraints	Validation	LOW	RESOLVED	p. 51
L-21	Missing EIP2612 Implementation	Code Best Practices	LOW	RESOLVED	p. 53

REMEDIATION REVIEW 1

M-01	Incorrect Accounting Due To _realizeProfit Call	Logical Error	MEDIUM	ACKNOWLEDGED	p. 54
M-02	PnL Realization Depends On Vault Activity	DoS	MEDIUM	ACKNOWLEDGED	p. 55
M-03	Missing Initialization of ERC20PermitUpgradeable	Logical Error	MEDIUM	RESOLVED	p. 56
M-04	Incorrect Fee-on-Fee Calculation In totalAssets Function	Logical Error	MEDIUM	RESOLVED	p. 57
L-01	Overcollected Redemption Fee Due To Price Impact	Warning	LOW	RESOLVED	p. 58
L-02	Unnecessary ERC2771 Check	Warning	LOW	RESOLVED	p. 59
L-03	Misleading Error	Warning	LOW	RESOLVED	p. 60
L-04	Incorrect Total Assets Cap Check	Logical Error	LOW	ACKNOWLEDGED	p. 62
L-05	ModifiedPosition event emits wrong referrer	Logical Error	LOW	RESOLVED	p. 64
L-06	ETH received via receive() can be locked	Warning	LOW	ACKNOWLEDGED	p. 65

REMEDIATION REVIEW 2

H-01	Incorrect Order Slippage Check	Validation	HIGH	ACKNOWLEDGED	p. 66
------	--------------------------------	------------	------	--------------	-------

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-01	Losses Perturb Delta Neutrality	Warning	MEDIUM	ACKNOWLEDGED	p. 67
M-02	Slippage in Swaps Can Prevent Flash Loan Repayment	DoS	MEDIUM	ACKNOWLEDGED	p. 68
L-01	Typo	Typo	LOW	RESOLVED	p. 69
L-02	Rebalance Calls Can Fail with ExceedsMarketCreditCapacity	DoS	LOW	ACKNOWLEDGED	p. 70

CRITICAL C-01

RESOLVED

Unrestricted Flash Loan Callback

CATEGORY	Validation
LOCATION	FundingRateVault.sol: 239
REVIEW	Main Review

DESCRIPTION

Within the `FundingRateVault`'s `executeOperation` function, the only validation performed is to check whether `msg.sender` matches the Aave Pool address. The function does not verify that the flash loan request was initiated by the vault itself, nor does it validate the contents of the params data beyond basic decoding. This opens a path for an attacker to call Aave's `flashLoanSimple` with the vault contract set as the `receiver`, thereby injecting arbitrary parameters for the vault's code to interpret as (`valueToRedeem`, `debt`). When the vault receives the flash loan callback, it proceeds to repay what it believes is its Synthetix debt, forcibly withdraw a matching amount of margin from Synthetix (`_removeMargin`), swap that withdrawn collateral to USDC and repay the flash loan principal plus the Aave premium. However, none of these operations were genuinely authorized by the vault's internal logic; they are triggered entirely by the attacker's call to Aave. As a result, the vault ends up paying the flash loan fee on every maliciously forced loan, and can have its position partially or fully unwound. This leads to a net depletion of the vault's margin and a direct financial cost to its depositors. By repeatedly forcing flash loans into the vault, an attacker can continually drain or damage the vault position without requiring a legitimate redemption flow or any vault involvement.

RECOMMENDATION

The `FundingRateVault` should enforce that it is the initiator of each flash loan → `initiator == address(this)`.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/58>

CRITICAL C-02

RESOLVED

Mismatch in the redemption mechanism

CATEGORY	Logical Error
LOCATION	FundingRateVault.sol
REVIEW	Main Review

DESCRIPTION

In Synthetix Perps V3, net positive PnL on a position is always credited to the account's snxUSD balance (`collateralId=0`), not in the originally deposited collateral type, see the following [SynthetixV3 docs] (<https://docs.synthetix.io/exchange/perps-v3-base/multi-collateral-margin/settlement-of-debt>). If the `FundingRateVault` started with `collateralId=5` (WETH-synth), all newly realized gains are booked as extra snxUSD margin rather than more WETH-synth. Meanwhile, the vault's redemption logic continues to call:

```
modifyCollateral(accountId, wethCollateralId, -int256(amountToRemove));
```

on the assumption that profits appear under `collateralId=5`. In reality, `collateralAmounts[wethCollId]` remains unchanged, so the contract will fail to redeem with an `InsufficientSynthCollateral()` error when it tries to withdraw those gains. Because no step converts the snxUSD margin (the actual profit) into WETH-synth, users effectively cannot redeem their realized PnL. The final outcome is that the vault's perps position can generate positive returns, but they're never accessible in WETH-synth form nor do the vault's calls to `modifyCollateral(..., 5, ...)` succeed once the margin surpasses the originally deposited WETH-synth.

RECOMMENDATION

Augment the vault to detect and swap the newly realized PnL out of snxUSD into WETH-synth before attempting to withdraw. For example, if the vault sees that `collateralId=0` (snxUSD) has accrued some realized profit, it must use the Spot Market to swap all the snxUSD into WETH-synth, thus increasing the vault's `collateralAmounts[wethCollId]` and therefore allowing users to `redeem` their actual share of the PNL accrued.

Without such swap, the vault is stuck calling `modifyCollateral` on the WETH-synth bucket for margin that actually lives under the snxUSD bucket, leaving users unable to redeem those realized gains.

C-02 Mismatch in the redemption mechanism

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/56>

HIGH **H-01**

ACKNOWLEDGED

Deposits Might Change Funding Rate Direction

CATEGORY	Warning
LOCATION	FundingRateVault.sol
REVIEW	Main Review

DESCRIPTION

The `FundingRateVault` strategy relies on the Synthetix Perpsv3 market having a net long skew (positive funding), where longs pay shorts and the vault's short position collects regular yield. When deposits surge, the vault's margin increases, leading to a larger short being opened. This additional short significantly raises the market's net short skew, edging the funding rate toward zero or eventually negative. If the funding rate becomes negative, the vault must pay longs instead of receiving yield, eliminating returns for depositors. Simultaneously, the vault's overall profit margin can be further eroded by multiple protocol fees (management fee, performance fee, keeper fee) and the slippage incurred during `USDC→cbETH→sCBETH` swaps as well as the Aave flash-loan fee paid whenever a redemption occurs. Taken together, these factors can cause the vault's yield to fall below zero if too many deposits push the perps market away from negative funding. This discourages user participation and can freeze liquidity additions if the vault must constantly pay positive funding.

RECOMMENDATION

Consider updating the `FundingRateVault` to limit incoming deposits when the market skew is close to neutral, or if the on-chain funding rate is negative (shorts pays longs). One approach is to monitor the perps market's skew and only accept new deposits while the funding rate remains sufficiently positive. Additionally, impose more restrictive deposit caps or dynamic gating so that large inflows do not abruptly flip the short's advantage into a liability. Finally, evaluate and disclose that even when the funding rate is negative, slippage on sizable swaps and recurring flash-loan overhead might overshadow the collected yield, especially if the vault's net funding advantage is only marginal.

RESOLUTION

Status: Acknowledged.

HIGH **H-02**

RESOLVED

_performanceFeeHighWaterMark Is Incorrectly Updated

CATEGORY	Logical Error
LOCATION	FundingRateVault.sol: 812
REVIEW	Main Review

— DESCRIPTION

In the current implementation, `_performanceFeeHighWaterMark` is reset unconditionally during the `_updatePerformanceFeeDebt` process, rather than only when the new exchange rate exceeds the old watermark. This leads to inaccurate performance fee calculations. If the share price is below the high-water mark but the vault still assigns `performanceFeeHighWaterMark = exchangeRate_`, the contract “resets” the baseline incorrectly and the next genuine increase charges fees on previously realized gains.

— RECOMMENDATION

Ensure the high-water mark updates only when the new exchange rate is strictly higher than the previous `_performanceFeeHighWaterMark`.

— RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/57>

HIGH **H-03**

RESOLVED

Realized PNL Breaks Delta Neutrality

CATEGORY	Logical Error
LOCATION	FundingRateVault.sol
REVIEW	Main Review

DESCRIPTION

When the `FundingRateVault`'s short position is profitable, Synthetix Perps V3 credits that net PnL to the vault's `snxUSD` balance (`collateralId=0`). This increases the vault's overall margin but does not increase its WETH-synth holding (`collateralId=5`). As a result, some fraction of the vault's collateral is now effectively stable (`snxUSD`), rather than reflecting the same ETH-based exposure that the vault originally intended on the collateral side. This partial switch to a stable token interrupts the vault's pure 1:1 offset between the short position size and the `sdbETH` collateral value. Because the stable portion has no sensitivity to the ETH price, the vault ends up partially net short or net long if the rebalancing logic still assumes all margin is in WETH-synth form. Over time, this mismatch will totally erode the vault's delta neutrality, leading to smaller or larger gains and losses than intended when ETH's price fluctuates. Essentially, the vault's design to remain price-neutral relies on maintaining an equal notional in WETH-synth and an opposite perps short, but storing realized profit in `snxUSD` breaks that symmetrical hedge.

RECOMMENDATION

Convert any `snxUSD`-based profit back into WETH-synth collateral as soon as it is realized. Without this conversion, the vault is forced into partial stable collateral and loses the strict delta neutrality it was designed to maintain.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/56>

HIGH **H-04**

RESOLVED

Fee-on-fee Calculation Inflates Vault Fees

CATEGORY	Logical Error
LOCATION	FundingRateVault.sol: 857-858, 824
REVIEW	Main Review

DESCRIPTION

The vault calculates management and performance fees based on the value returned by `_totalAssetsExcludingDebt()`. This function returns the total assets within the Synthetix account (`_availableMargin`) and the `USDC` held directly by the vault contract (`idleAssets`).

However, accrued management fees (`_managementFeeDebt`) and performance fees (`_performanceFeeDebt`) exist as internal liabilities intended to be paid from the returned `_totalAssetsExcludingDebt` value. Because `_totalAssetsExcludingDebt()` does not subtract these accrued fee liabilities, the base value used for fee calculation is inflated by the amount of unpaid fees.

1. In `_getManagementFeeDebt`, the `managementFee` percentage is applied to `_totalAssetsExcludingDebt()`, incorrectly charging management fees on previously accrued, unpaid fee amounts resident within returned `_totalAssetsExcludingDebt` value.
2. In `_getPerformanceFeeDebt`, the `_exchangeRate` used for High-Water Mark comparison and `perShareGain` calculation is derived from `_totalAssetsExcludingDebt()`. This inflates the perceived asset value per share, potentially leading to performance fees being charged on a slightly larger base than if calculated purely on principal + trading PnL. Furthermore this could lead to `_performanceFeeHighWaterMark` being updated due to highly accrued unpaid fee liabilities (As it would increase the `_exchangeRate` calculation).

This results in a "fee-on-fee" scenario where users are overcharged over time.

RECOMMENDATION

Modify the fee calculation logic to calculate the management fee (`_getManagementFeeDebt`) and performance fee (`_getPerformanceFeeDebt`) based on `_totalAssetsExcludingDebt()` minus unpaid `_managementFeeDebt` and `_performanceFeeDebt`. This ensures the fee calculation is based solely on the principal assets and trading PnL, excluding already accrued, unpaid fee liabilities.

H-04 Fee-on-fee Calculation Inflates Vault FeesCONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/55>

MEDIUM **M-01**

RESOLVED

Potential Underflow in totalAssets Function

CATEGORY	Logical Error
LOCATION	FundingRateVault.sol: 228
REVIEW	Main Review

DESCRIPTION

The `totalAssets` function subtracts the vault's outstanding management fee and performance fee debts from the vault's raw asset balance (`_totalAssetsExcludingDebt`). If `managementFeeDebt + performanceFeeDebt` surpasses that excluding-debt amount, the subtraction underflows in Solidity 0.8+, triggering an immediate revert. Since both `exchangeRate` and user deposit/redeem flows rely on `totalAssets`, any call that references this function also reverts, effectively locking the vault. The only escape is to send raw USDC directly into the contract (bypassing normal deposit logic) to restore a non-negative net asset balance and prevent the underflow. Until that happens, depositors cannot deposit or redeem, leaving the vault in an unusable state.

RECOMMENDATION

Implement a safeguard that prevents fee debt from ever exceeding the vault's raw balance, or handle the case by capping total outstanding fees at `_totalAssetsExcludingDebt`. One approach is to allow the contract to zero out or partially collect fees when the vault's assets are too low, rather than letting the subtraction underflow. This ensures that `totalAssets` can always compute a non-negative result, avoiding a scenario where the vault becomes permanently stuck unless externally rescued with a direct USDC transfer.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/53>

MEDIUM **M-02**

ACKNOWLEDGED

Excessive Swap Amounts Risk Failing

CATEGORY Warning
LOCATION FundingRateVault.sol: 606
REVIEW Main Review

DESCRIPTION

The `FundingRateVault`'s `_swapAll` calls convert the entire deposit or redemption amount from USDC → WETH, and then WETH → cbETH or cbBTC. The code enforces no slippage constraints, meaning `amountOutMinimum=0`, so front-runners can sandwich the vault's trade, inflating user costs or draining vault value. Additionally, the current `maxAssetTransactionSize` is set at 500,000 USDC. If the target AMM pool (e.g. the Aerodrome or Uniswap V3-style pool) holds less total liquidity than needed for a near-1:1 swap at the moment, the swap may fail or produce severe price impact. For example, if the WETH/cbETH pool only has ~1,372 cbETH or 928 WETH of real depth, a 500,000 USDC deposit converted into WETH and then swapped may exceed the feasible trading capacity, leading the router to revert or yield an extremely low amount of the final token. In a best-case scenario, the vault obtains fewer tokens due to the large price movement; in a worst case, the AMM fails to execute the swap altogether. Users are totally exposed to the losses caused by this price impact/slippage and moreover, they would have to go through the swaps once again (in the opposite direction) upon redemption. Basically, a user depositing 500,000 USDC would never get any profit as the price impact and slippage that he would take during the deposit and redemption would be way higher than the `FundingRateVault` realistic APY.

RECOMMENDATION

Consider decreasing the `maxAssetTransactionSize` to a value much lower. On the other hand, consider allowing the users to provide directly the cbBTC/cbETH during deposits or let them pass a `minAmountOut` for the two `_swapAll` calls in order to prevent slippage.

Finally, consider monitoring in real-time the pools liquidity to adjust dynamically `maxAssetTransactionSize`.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

RESOLVED

Repeated Tiny Deposits

CATEGORY Warning
LOCATION FundingRateVault.sol: 772
REVIEW Main Review

DESCRIPTION

Because every deposit triggers an async order on Synthetix, the `FundingRateVault`'s `_validateNoPendingOrder` reverts if another order has not yet settled or been cancelled. This means only one deposit or redemption can be initiated per block. An attacker (or "griefer") can exploit this by sending repeated dust-sized deposits, paying a small keeper fee per transaction (about \$1 plus minimal gas), thereby leaving the vault in a perpetual "pending order" state. Given Base's short block times (~2 seconds), a malicious user could spam these dust deposits, each creating a new async order every block, effectively preventing any other depositor from interacting with the vault. The costs are not prohibitively high for a determined attacker; just around 30 USD per minute to keep other users locked out.

RECOMMENDATION

Consider enforcing a minimum deposit/redemption amount.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/59>

MEDIUM **M-04**

RESOLVED

Missing Proper Slippage Checks

CATEGORY Code Best Practices
LOCATION FundingRateVault.sol
REVIEW Main Review

DESCRIPTION

In the `FundingRateVault`'s deposit and redemption flows, the code executes `_swapAll(USDC, WETH, ...)` followed by `_swapAll(WETH, cbETH/cbBTC, ...)` without specifying any `amountOutMinimum` or `slippage` buffer. As a result, these large, single-transaction swaps can be easily front-run by MEV bots. A bot can detect the vault's transaction in the mempool, buy up the token that the vault intends to purchase (pushing the price higher), let the vault pay a worse price, and then sell immediately after at a profit, effectively "sandwiching" the vault's trade. Because `_swapAll` passes `amountOutMinimum=0`, the vault has zero recourse to revert if the price is far worse than expected, opening a path for repeated front-running.

RECOMMENDATION

Include meaningful slippage protection when calling `_swapAll(...)`, specifying a nonzero `amountOutMinimum` that reflects a maximum price impact the vault is willing to tolerate. This `amountOutMinimum` should be given for each swap call and passed as parameter by the user to the `deposit` and `redeem` functions.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/54>

MEDIUM **M-05**

ACKNOWLEDGED

Redeem Fail When Very Low Total Deposits

CATEGORY Logical Error
LOCATION FundingRateVault.sol: 629
REVIEW Main Review

DESCRIPTION

When the vault's total deposited assets are very low (e.g., under \$100), the `redeem` function can revert if it tries to withdraw too much collateral. Although the vault enforces a `maxRedemptionPercent`, it overlooks that Synthetix Perps requires a minimum amount of margin (the `requiredInitialMargin`) to remain posted on the account. In a proof of concept, a `requiredInitialMargin` of about \$22 caused `modifyCollateral` in the `redeem` flow to revert if the vault attempts to withdraw more margin than is permissible under that minimum. This means the vault can't practically allow redemption of a large share portion when the total vault size is too small, making it impossible for certain users to exit.

RECOMMENDATION

The owner should seed the vault with enough initial collateral to satisfy the perps market's minimum margin requirements. This ensures that, even if the vault's total assets are relatively small, redemptions won't force the position to drop below the `requiredInitialMargin` and trigger a revert. A simple fix is to deposit a baseline amount upon vault initialization so that the margin can cover smaller user deposits and still remain above the Synthetix Perps threshold.

PROOF OF CONCEPT <https://github.com/GuardianOrg/vaults-contracts-team1/blob/e59f4bb/b9c4ab7dc020ceed551d534d77d02fe3a/test/guardian/pocs/redeemFail.t.sol>

RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

RESOLVED

Potential DoS via Unexpected ETH

CATEGORY	Logical Error
LOCATION	FundingRateVault.sol: 606
REVIEW	Main Review

DESCRIPTION

In the vault's `_swapAll` function, the contract invokes `exactInputSingle` on Aerodrome. Internally, Aerodrome's swap logic may call `refundETH` at the end of the swap, which sends any leftover ETH balance in Aerodrome back to the `msg.sender` (in this case, the `FundingRateVault`). Typically, there should be no raw ETH in Aerodrome, since WETH is used for trades. However, a malicious actor could deposit ETH into Aerodrome via a `selfdestruct`, leaving unexpected ETH behind that triggers a `refundETH` call. If the `FundingRateVault` cannot handle receiving this ETH (e.g. no `receive` or fallback implemented), the transaction will revert, blocking the vault from completing the swap or proceeding with further logic.

RECOMMENDATION

Implement a minimal `receive` or `fallback` function in the `FundingRateVault` that safely accepts stray ETH refunds.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/52>

MEDIUM **M-07**

ACKNOWLEDGED

Leftover SNX_USD are stuck in the contract

CATEGORY Warning
LOCATION FundingRateVault.sol: 798-809
REVIEW Main Review

DESCRIPTION

Once the `_swapUsdcForSnxUsd` is called. The goal is to repay entire debt amount first. However, very often there will be more sUSD swapped than the required debt amount and the leftover amount of `SNX_USD` will accumulate and stuck in the contract.

RECOMMENDATION

The way to unlock these extra `SNX_USD` amount must be implemented

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Unnecessary Zero-Fee Transfers

CATEGORY Gas Optimization
LOCATION FundingRateVault.sol: 901
REVIEW Main Review

DESCRIPTION

In the `_payDepositFee` and `_payRedemptionFee` functions, the contract calculates the fee as `amount.mul(depositFee or redemptionFee)`, then blindly calls `_payFees(fee)` even if `fee` is zero. While this may not break functionality, it triggers a token transfer to the `feesRecipient` for a zero amount, which is an extra step with no real effect and increases the gas costs.

RECOMMENDATION

Add the following check before calling `_payFees(fee)`:

```
if (fee > 0) {  
  _payFees(fee);  
}
```

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/51>

LOW **L-02**

ACKNOWLEDGED

Excessive Gas Usage in Deposit Flow

CATEGORY	Warning
LOCATION	FundingRateVault.sol: 618, 629
REVIEW	Main Review

— DESCRIPTION

During testing of the `deposit` function, it was observed that each deposit triggers a `perpsMarket.modifyCollateral` call, which in turn invokes `MarketCollateralModule.depositMarketCollateral`. Internally, this logic calls `marketData.getReportedDebt` (and in turn the `reportedDebt(perpsMarketId)` function on the Perps contracts). The `reportedDebt` implementation iterates over all active markets to compute total debt (invoking price lookups and summations). This results in an extremely high gas overhead, as every single deposit includes a chain of calls that sum all active markets' debts and collateral values. In practice, the combined depth of delegate calls and loops over all markets can consume over seven million gas for a single deposit transaction.

An example user transaction shows that using `modifyCollateral` on a single market can consume up to seven million gas. This overhead will only worsen if new markets or new loops appear in `reportedDebt`.

This also affects `redeem` calls as during the redemption a call to `_perpsMarket.modifyCollateral` is also performed.

— RECOMMENDATION

There is no straightforward solution to this. The only solution would be reducing the amount of active Synth markets that support USDC as collateral.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Lack Of ERC4626 View Functions

CATEGORY	Warning
LOCATION	FundingRateVault.sol
REVIEW	Main Review

— DESCRIPTION

While the `FundingRateVault` is intended to follow the `ERC4626` standard, it does not implement certain key view functions such as `previewDeposit(uint256 assets)` and `previewRedeem(uint256 shares)`. These methods are part of the core `ERC4626` specification to allow users (or other contracts) to query the expected share output for a given deposit, or to query the expected asset return for a given share redemption, before executing the actual transaction.

Without these `preview*` functions, integrators have no on-chain mechanism to simulate how many vault shares would be minted by a deposit, or how many underlying assets would be returned by a redemption. This deviates from the `ERC4626` standard. In turn, users have to rely on off-chain estimates or attempt static calls to the actual deposit/redeem logic, which may not align with the explicit standard.

— RECOMMENDATION

Implement the standard `ERC4626` "preview" methods, such as `previewDeposit(uint256 assets)` and `previewRedeem(uint256 shares)`, returning accurate estimates without side effects or reverts. This will let integrators (wallets, front-ends or other smart contracts) rely on the canonical `ERC4626` interface to discover the share/asset exchange ratio in real time, enhancing compatibility and compliance with the specification.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Lack Of a Double Step Transfer Ownership Pattern

CATEGORY Code Best Practices
LOCATION FundingRateVault.sol
REVIEW Main Review

DESCRIPTION

The standard [OpenZeppelin's Ownable contract](https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/access/Ownable.sol) allows transferring the ownership of the contract in a single step:

```
/**
 * @dev Transfers ownership of the contract to a new account ('newOwner').
 * Can only be called by the current owner.
 */
function transferOwnership(address newOwner) public virtual onlyOwner {
    if (newOwner == address(0)) {
        revert OwnableInvalidOwner(address(0));
    }
    _transferOwnership(newOwner);
}

/**
 * @dev Transfers ownership of the contract to a new account ('newOwner').
 * Internal function without access restriction.
 */
function _transferOwnership(address newOwner) internal virtual {
    address oldOwner = _owner;
    _owner = newOwner;
    emit OwnershipTransferred(oldOwner, newOwner);
}
```

If the nominated EOA account is not a valid account, it is entirely possible that the owner may accidentally transfer ownership to an uncontrolled account, losing the access to all functions with the `onlyOwner` modifier.

L-04 Lack Of a Double Step Transfer Ownership Pattern

CONTINUED

RECOMMENDATION

It is recommended to implement a two-step transfer process in the `FundingRateVault` contract where the owner nominates an account and the nominated account needs to call an `acceptOwnership` function for the transfer of the ownership to fully succeed. This ensures the nominated EOA account is a valid and active account. A good code example could be [OpenZeppelin's Ownable2Step] (<https://github.com/OpenZeppelin/openzeppelin-contracts/blob/master/contracts/access/Ownable2Step.sol>) contract:

```
/**
 * @dev Starts the ownership transfer of the contract to a new account. Replaces the pending
 * transfer if there is one.
 * Can only be called by the current owner.
 */
* Setting 'newOwner' to the zero address is allowed; this can be used to cancel an initiated
ownership transfer.
*/
function transferOwnership(address newOwner) public virtual override onlyOwner {
    _pendingOwner = newOwner;
    emit OwnershipTransferStarted(owner(), newOwner);
}

/**
 * @dev Transfers ownership of the contract to a new account ('newOwner') and deletes any
 * pending owner.
 * Internal function without access restriction.
 */
function _transferOwnership(address newOwner) internal virtual override {
    delete _pendingOwner;
    super._transferOwnership(newOwner);
}

/**
 * @dev The new owner accepts the ownership transfer.
 */
function acceptOwnership() public virtual {
    address sender = _msgSender();
    if (pendingOwner() != sender) {
        revert OwnableUnauthorizedAccount(sender);
    }
    _transferOwnership(sender);
}
```

L-04 Lack Of a Double Step Transfer Ownership PatternCONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

Deposits May Revert

CATEGORY	Validation
LOCATION	FundingRateVault.sol: 621, 632
REVIEW	Main Review

DESCRIPTION

During the `FundingRateVault`'s deposit flow, the user's USDC is swapped into WETH or another asset, and multiple fees (keeper fee, deposit fee, slippage from swaps) will be subtracted before `_depositMargin` is called. If these combined fees consume the entire deposit, the resulting balance for `_depositMargin` can be zero, causing the vault's call to `modifyCollateral(accountId, collateralId, int256(balance))` to revert. Consequently, user deposits that exactly cover or barely exceed the fees end up reverting, blocking smaller or fee-burdened deposits that the user expects to go through.

RECOMMENDATION

Check whether the final balance is zero (or below a threshold) before calling `modifyCollateral` and skip the margin deposit if no net collateral remains. For example:

```
uint256 balance = <result of swap minus fees>;
if (balance > 0) {
  // only then call _depositMargin
  _perpsMarket.modifyCollateral(accountId, collateralId, int256(balance));
}
```

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/50>

LOW **L-06**

ACKNOWLEDGED

Last User Can Not Fully Redeem

CATEGORY Logical Error
LOCATION FundingRateVault.sol
REVIEW Main Review

— DESCRIPTION

The `FundingRateVault` enforces a hard-coded `MAX_REDEMPTION_PERCENT = 0.5e18`, preventing any single redemption from exceeding 50% of the vault's total assets in one transaction. If only one user remains and wants to redeem all their shares in one go, they will be blocked, since attempting to redeem more than 50% will revert. The user is forced into a cycle of partial redemptions—redeeming 50%, then 50% of the remainder, etc. This is effectively an infinite process because after each successful redemption, the vault's assets keep shrinking, and the user still holds some fraction above 50% of that new supply. Meanwhile, if the vault is effectively shutting down, the user should be able to redeem everything at once.

Additionally, leaving even a small portion of the Perps V3 position open means the vault continues paying or receiving funding, incurring fees until the user manually repeats partial redemptions. This is not intuitive to end-users expecting to withdraw in a single final transaction.

— RECOMMENDATION

Allow the last user to bypass the 50% redemption cap if they hold all the remaining supply. One approach is to detect when a redeem request matches the total share supply, then fully close the Perps V3 position (settle or repay any debt) and distribute all assets in a single transaction. This ensures that a final "all-in" redemption can happen without forcing repeated partial transactions, preventing frustrating user experiences and high on-chain overhead.

— RESOLUTION

Status: Acknowledged.

<https://github.com/Synthetixio/vaults-contracts/pull/49>

LOW **L-07**

ACKNOWLEDGED

Flashloan Fee Is Not Accounted

CATEGORY Warning
LOCATION FundingRateVault.sol: 461
REVIEW Main Review

— DESCRIPTION

When the `FundingRateVault` initiates a flash loan in the `realizeFees` function to free margin from Synthetix Perps (e.g., repaying the short's debt before withdrawing collateral), it incurs a flash loan fee that is paid to Aave. However, this fee is never accrued or tracked in any "pending fees" variable within the vault's accounting. As a result, the vault's `totalAssets` remains a bit higher than it should, ignoring the future obligation to pay the flash loan fee when the fees are realized. When `realizeFees` is called, the `FundingRateVault` pays that untracked flash loan fee out of its newly withdrawn collateral, reducing the net assets of the vault and causing a sudden drop in `totalAssets`, a hidden loss for depositors who assumed the vault's net asset figure included all liabilities. This discrepancy can be particularly acute if the debt repaid in the flash loan is high or the admins frequently call the `realizeFees` function.

— RECOMMENDATION

Treat the flash loan fee as an expense covered by the vault's collected fees, rather than by user assets. Whenever `realizeFees` is called compute the fee premium from Aave and temporarily charge it to the accrued management/performance fees.

— RESOLUTION

Status: Acknowledged.

LOW **L-08**

RESOLVED

Unset Referrer Field

CATEGORY	Warning
LOCATION	FundingRateVault.sol
REVIEW	Main Review

DESCRIPTION

When the `FundingRateVault` calls `PerpsMarketProxy.commitOrder(commitment)`, it fills in `referrer = address(0)` by default. Synthetix Perps V3 supports an optional referral mechanism that rewards a specified address (the referrer) with a share of fees or other incentives for order flow. By always setting `referrer = address(0)`, the `FundingRateVault` forfeits any potential fee rebates or revenue share that could accrue through a recognized referrer. This omission might mean the vault (or its admins) lose out on additional protocol benefits, or at least do not leverage Synthetix's referral programs to its advantage.

RECOMMENDATION

Consider passing a valid `referrer` address if the vault or its operators wish to participate in Synthetix's referral or integrator reward system.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/48>

LOW **L-09**

RESOLVED

Vault Is More Exposed To Liquidation

CATEGORY Warning
LOCATION FundingRateVault.sol
REVIEW Main Review

DESCRIPTION

When the `FundingRateVault` processes a redemption, it immediately withdraws some collateral from Synthetix Perps to deliver assets to the user, yet it does not simultaneously adjust its short position. Until the vault's new async order is executed (to rebalance or resize the short), the vault's margin is lower while its short notional remains the same. This means the account is closer to the liquidation threshold, any adverse price move or funding rate expense in this brief window can push the vault into liquidatable territory. Although this interval may last only as long as Synthetix's settlement delay (often a few blocks), it represents a transient but real risk where the vault's short becomes under-collateralized. Users should be aware that, during this small period, a sudden negative price movement could lead to liquidation before the short is adjusted.

RECOMMENDATION

Document that each redemption introduces a short-lived window of increased liquidation risk until the vault's rebalancing order is settled.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/37>

LOW **L-10**

ACKNOWLEDGED

Admin Should Call rebalance Frequently

CATEGORY Warning
LOCATION FundingRateVault.sol: 445
REVIEW Main Review

DESCRIPTION

The `FundingRateVault` relies on a short Synthetix Perps position offsetting its spot (cbETH or cbBTC) holdings. However, it only adjusts that short during deposit and redemption flows or when an admin explicitly calls `rebalance`. In periods of large market movement or after deposit/redemption inactivity, the vault's net margin and short notional can drift away from the intended 1:1 hedge. Over time, even minor discrepancies can result in unexpected directional exposure, leaving the vault (and its users) open to losses from price swings and missing out on accurate negative funding collection. By default, if no user deposit or redemption triggers rebalancing, the short may remain at an outdated size while the underlying asset's price moves significantly.

RECOMMENDATION

Ensure that an admin (or automated keeper process) calls `rebalance` at regular intervals, particularly after significant market volatility or extended inactivity in deposits/redemptions.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Theoretical First-Depositor Inflation Attack

CATEGORY	Warning
LOCATION	FundingRateVault.sol
REVIEW	Main Review

— DESCRIPTION

In ERC4626-style vaults, there is a known “first depositor inflation attack” scenario where an attacker who deposits a tiny amount first could mint an outsized fraction of total shares, later diluting legitimate participants who join once the vault’s net asset value grows. In the `FundingRateVault`, this exploit would ordinarily allow the attacker to extract disproportionate profits. However, the vault’s design imposes price impact calculations and fees on each deposit, meaning that any attempt to artificially understate the vault’s initial total asset base or game the share price is corrected by the swap slippage and fee overhead. Consequently, the first depositor is very unlikely to mint shares at a wildly favorable ratio and walk away with inflated ownership. The combination of slippage cost, deposit fees and the subsequent position rebalancing effectively negates the standard first-depositor exploit.

— RECOMMENDATION

Although the vault’s logic disincentivizes a trivial inflation attack, it is still prudent to perform a small “seed deposit” by a trusted party before opening the vault to public deposits. This ensures no user can claim a near-zero total asset base. The contract’s swap-based deposit process and fee structure already prevent a catastrophic dilution scenario, but initializing the vault with a nominal deposit would be an extra safeguard.

— RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

Order Cancellation Exposes Vault To Market

CATEGORY Warning
LOCATION FundingRateVault.sol: 662
REVIEW Main Review

— DESCRIPTION

Deposit order cancellation instead of settlement, that can happen during a very volatile market in case market rapidly moves against the order direction, and breaks the slippageBuffer, will leave the vault exposed to price fluctuations.

Until a new _rebalancePosition is called, the vault will not sit in a delta neutral state, making it exposed to the market, and possibly resulting in loss.

— RECOMMENDATION

Consider implementing a bot that constantly checks for the possibility to rebalance the vault position in order to keep is always delta-neutral.

— RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Missing Max. Open Interest Checks

CATEGORY	Validation
LOCATION	FundingRateVault.sol: 131-157
REVIEW	Main Review

DESCRIPTION

When a new order is being settled, there are some checks that can prevent the settlement. Some of these checks are for maximum open interest in both tokens and USD value.

```
function validatePositionSize(
    Data storage self,
    uint256 maxSize,
    uint256 maxValue,
    uint256 price,
    int128 oldSize,
    int128 newSize
) internal view {
    bool isReducingInterest = MathUtil.isSameSideReducing(oldSize, newSize);
    if (!isReducingInterest) {
        ...
        if (maxSize < MathUtil.abs(newSideSize / 2)) {
            revert PerpsMarketConfiguration.MaxOpenInterestReached();
        }
        if (maxValue > 0 && maxValue < MathUtil.abs(newSideSize / 2).mulDecimal(price)) {
            revert PerpsMarketConfiguration.MaxUSDOpenInterestReached();
        }
    }
}
```

Basically when the order is increasing the short position it ensures that the total open interest of the market doesn't exceed a specific limit. This can lead a state where deposits from users enter the spot market but the vault can't rebalance the position because the max open interest is already reached and the order of the position can't be settled. This situation would fail to keep the delta neutral position

L-13 Missing Max. Open Interest Checks**CONTINUED**

— RECOMMENDATION

When any of these limits is reached in the market, the vault should stop allowing the execution of new deposits to maintain the delta neutral position.

— RESOLUTION

Status: Acknowledged.

LOW **L-14**

RESOLVED

Debt Can Be Artificially Inflated

CATEGORY Warning
LOCATION FundingRateVault.sol: 662
REVIEW Main Review

— DESCRIPTION

Attacker can make a lot of small deposits and artificially increase the debt of the Vault position.

The debt increases every time the settlement is made.

Thus, attacker can make a multiple deposit and affect the already existing depositors, by diminishing the amount that they will redeem.

— RECOMMENDATION

I advise to make the check related of the max.possible debt that can be accrued, if some % of debt from the current total is accrued , the debt repayment must take place

PROOF OF CONCEPT <https://www.notion.so/Debt-can-be-artificially-inflated-harming-protocol-1c0b0257612480859405dcdfea05f1d4?pvs=4>

— RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/47>

LOW **L-15**

ACKNOWLEDGED

Ineffective Swap Deadline

CATEGORY Validation
LOCATION FundingRateVault.sol: 600
REVIEW Main Review

DESCRIPTION

The `_swapAll` function in `FundingRateVault.sol` uses `block.timestamp` as the `deadline` for Aerodrome swaps. This is ineffective and provides no protection against slippage or transaction delays.

The Aerodrome `ISwapRouter` interface (and most DEX routers) includes a `deadline` parameter in swap functions. If the transaction isn't executed before this deadline, the swap reverts. This protects against:

1. Excessive Slippage: If the market moves significantly between the time the transaction is submitted and when it's executed, the user might receive a much worse price than expected.
2. Transaction Delays: If the transaction gets stuck in the mempool for a long time, the price might change significantly.

By setting `deadline` to `block.timestamp`, the vault is essentially saying, "This transaction must execute in this block or revert." While this seems like a strict constraint, it's actually meaningless in practice. Since the vault's transaction itself is already part of the current block, `block.timestamp` will always be equal to the block's timestamp when `exactInputSingle` is called. The `deadline` check within Aerodrome's router (`require(block.timestamp <= params.deadline)`) will always pass. It will never revert due to the deadline. As such there is not protection against slippage or delayed transaction execution, during the `_swapAll` function.

The `amountOutMinimum` is also set to 0. Which further eliminates the slippage check.

RECOMMENDATION

The `deadline` should be a configurable parameter passed in as an user input parameter. Should not be set to `block.timestamp`.

L-15 Ineffective Swap Deadline

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

LOW **L-16**

ACKNOWLEDGED

Redemption Failure Due To Aave State

CATEGORY	Warning
LOCATION	FundingRateVault.sol: 190, 461
REVIEW	Main Review

DESCRIPTION

The `redeem` function in `FundingRateVault.sol` is vulnerable to failure if the Aave USDC reserve becomes paused, inactive, or has flash loans disabled. This contradicts the vault's documentation, which states that "Redemptions should always be enabled." The issue stems from the `redeem` function's reliance on an Aave flash loan for paying out debt.

Here's the relevant flow:

1. `redeem`: A user requests to redeem their shares.
2. `Existing Debt`: If the vault account has existing debt, it attempts to obtain the necessary USDC via an Aave flash loan to repay the debt amount.
3. `Flash Loan Initiation`: The code uses `_usdc.flashLoanSimple(...)` to initiate the flash loan.
4. `validateFlashloanSimple`: Inside the aUSDC's contract, the following require statements are executed.

```
require(!configuration.getPaused(), Errors.RESERVE_PAUSED);  
require(configuration.getActive(), Errors.RESERVE_INACTIVE);  
require(configuration.getFlashLoanEnabled(), Errors.FLASHLOAN_DISABLED);
```

If the Aave USDC reserve is paused, inactive, or has flash loans disabled, the `_usdc.flashLoanSimple(...)` call will revert due to the `validateFlashloanSimple` check within Aave's `aUSDC` contract. This revert will propagate and cause the entire `redeem` transaction to fail, effectively blocking redemptions. Hence this introduces a single point of failure in the vault's redemptions through an external party (Aave).

RECOMMENDATION

It is recommended to implement a fallback mechanism to repay the debt and execute the redeem. This can be achieved by implementing a backup `flashloan` mechanism via a different protocol which provides flashloan capability. Else it is recommended to introduce admin function to change the `flashloan` contract from `Aave` to another `protocol` in the event `Aave` USDC flashloan capability is paused or disabled.

L-16 Redemption Failure Due To Aave State

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/Synthetixio/vaults-contracts/pull/46>

LOW **L-17**

RESOLVED

Precision Loss in Redemption Calculation

CATEGORY	Logical Error
LOCATION	FundingRateVault.sol: 908, 909
REVIEW	Main Review

— DESCRIPTION

The `FundingRateVault::_valueToRedeem` function calculates the amount of underlying assets a user should receive based on the shares they are redeeming. The current implementation performs division before multiplication:

```
uint256 share = shares.div(totalSupply());  
uint256 assetsShare = totalAssets().mul(share);
```

Due to Solidity's integer arithmetic, the division `shares.div(totalSupply())` truncates any fractional part of the result. This intermediate `share` variable may therefore represent a slightly lower proportion than the user's true ownership, especially when `shares` is not perfectly divisible by `totalSupply()`.

This precision loss, introduced by the division, is then potentially magnified when multiplied by `totalAssets()`. Consequently, the calculated `assetsShare` might be slightly less than the amount the user is proportionally entitled to. While the effect might be small depending on the numbers involved, it represents a potential loss of value for the redeeming user due to rounding errors.

The rounding should happen in favour of the protocol. But here the rounding down error (due to division before multiplication) introduced is comparatively more. If we perform the multiplication followed by division still the rounding will happen in favor of the protocol with less precision loss to the redeeming user.

— RECOMMENDATION

To minimize precision loss and ensure a more accurate calculation of redeemed assets, perform multiplication before division. This preserves intermediate precision better in integer arithmetic.

Refactor the calculation as follows:

```
// Calculate assetsShare = (totalAssets() * shares) / totalSupply()  
uint256 assetsShare = totalAssets().mul(shares).div(totalSupply());
```

L-17 Precision Loss in Redemption Calculation

CONTINUED

This revised order ensures that the full precision of `totalAssets().mul(shares)` is maintained before the final division occurs, leading to a more accurate `assetsShare` value.

— RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/45>

LOW **L-18**

ACKNOWLEDGED

Cache Total Fee Amount

CATEGORY	Gas Optimization
LOCATION	FundingRateVault.sol: 143-146
REVIEW	Main Review

— DESCRIPTION

There are some different fees that are paid during different executions. However, in each method that requires to pay for fees it executes a different ERC20 transfer for each fee. As an example, in the deposit function, it executes 4 different ERC20 transfers when it could compute all fees added and execute a single transfer to save gas. Notice that each transfer triggers a state change.

```
function deposit(
  uint256 amount,
  address receiver,
  uint256 minShares
) public override returns (uint256 shares) {
  ...
  _payKeeperFee();
  _payDepositFee(amount);
  _payManagementFeeDebt();
  _payPerformanceFeeDebt();
  ...
}
```

— RECOMMENDATION

Cache the total fee amount that will be sent to the fee receiver and only execute a single ERC20 transfer

— RESOLUTION

Status: Acknowledged.

LOW **L-19**

RESOLVED

maxRedeem Function Ignores maxRedemptionPercent

CATEGORY	Warning
LOCATION	FundingRateVault.sol: 575-584
REVIEW	Main Review

DESCRIPTION

The `maxRedeem` function is meant to provide users with a method to determine the maximum amount of shares they can redeem in exchange for the underlying token. However, this function does not take into account the maximum redemption percent when computing this amount. This can lead to a user fetching this method to get how much shares he can redeem and failing during the execution because of the maximum redemption allowed.

RECOMMENDATION

Take into account the percent that the owner holds and cap it at the maximum redemption percent.

PROOF OF CONCEPT <https://github.com/GuardianOrg/vaults-contracts-team1/pull/4/files>

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/44>

LOW **L-20**

RESOLVED

Missing Fee Setting Constraints

CATEGORY	Validation
LOCATION	FundingRateVault.sol: 348-391
REVIEW	Main Review

DESCRIPTION

There are specific fees that are meant to be a percentage of a certain amount such as the `depositFee`, `performanceFee` or the `redemptionFee`. However, the function to set these values does not have any validation to be in an accepted range:

```
function updateDepositFee(
    uint256 newDepositFee
) external override onlyOwner {
    if (newDepositFee == depositFee) revert InvalidValue();
    depositFee = newDepositFee;
    emit DepositFeeUpdated(newDepositFee);
}
```

They only ensure that the new value is different from the previous one. That means that if the owner messes with a specific fee by setting it out of the expected range it can block core functionalities such as the `deposit` function.

As an example, if the owner sets the `depositFee` to 1.1 ether which represents 110% when someone will try to call the deposit function it will fail because the `_payDepositFee` function will fail because the contract does not hold 110% of assets that the user sent to the contract:

```
function _payDepositFee(uint256 amount) internal {
    // (amount * depositFee) / 10 ** 18;
    uint256 fee = amount.mul(depositFee);
    _payFees(fee);
    emit DepositFeesPaid(fee);
}

function _payFees(uint256 amount) internal {
    _usdc.transfer(feesRecipient, amount);
    emit FeesPaid(amount);
}
```

L-20 Missing Fee Setting Constraints**CONTINUED**

— RECOMMENDATION

Constrain fee setting to be in the expected range:

```
function updateDepositFee(
  uint256 newDepositFee
) external override onlyOwner {
  ++ if (newDepositFee > 1 ether) revert InvalidValue();
  if (newDepositFee == depositFee) revert InvalidValue();
  depositFee = newDepositFee;
  emit DepositFeeUpdated(newDepositFee);
}
```

Notice that this must be done with all fees that are meant to be a percentage.

— RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/43>

LOW **L-21**

RESOLVED

Missing EIP2612 Implementation

CATEGORY Code Best Practices
LOCATION FundingRateVault.sol
REVIEW Main Review

DESCRIPTION

According to the ERC4626, tokenized vaults may implement the EIP2612 to improve the UX of approving shares on various integrations.

RECOMMENDATION

Implement the EIP2612 to improve user experience and have a higher compatibility with ERC4626 standard.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/42>

MEDIUM **M-01**

ACKNOWLEDGED

Incorrect Accounting Due To `_realizeProfit` Call

CATEGORY Logical Error
LOCATION `FundingRateVault.sol`
REVIEW Remediation Review 1

DESCRIPTION

The `FundingRateVault`'s `_realizeProfit` function converts unrealized SNXUSD profits into real collateral by unwrapping sUSDC into USDC, then swapping USDC to WETH and WETH to cbWETH. If there is significant slippage in these swaps, the final cbWETH tokens received can be noticeably lower than what the vault previously counted as theoretical profit. As a result, a user viewing `totalAssets` before `_realizeProfit` would see a higher figure (assuming ideal prices), but once `_realizeProfit` completes, `totalAssets` might drop due to slippage.

In both the `deposit` and `redeem` flows, the contract takes a snapshot of `totalAssets` (or a derivative value like `_valueToRedeem`) before it calls `_realizeProfit`. However, `_realizeProfit` will shift the vault's `totalAssets` once it converts leftover SNXUSD into real collateral, because of the slippage that happens in the process during the USDC to WETH and WETH to cbWETH swaps. This means the deposit/redeem calculations are performed against a stale, pre-profit-realization figure, leading to inaccuracies in the number of shares minted or the USDC redeemed. If the actual final `totalAssets` is lower after `_realizeProfit`, the vault will over-mint shares or allow excessive redemptions.

RECOMMENDATION

The easiest mitigation is to adjust both the `FundingRateVault`'s internal accounting and user-facing calls so that any deposit or redemption logic relies on the fully realized state. To achieve this, consider calling `_realizeProfit` early in the transaction, recalculate `totalAssets` and only then proceed with deposit/redeem math, ensuring the share price is accurate.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

PnL Realization Depends On Vault Activity

CATEGORY DoS
LOCATION FundingRateVault.sol
REVIEW Remediation Review 1

DESCRIPTION

This issue occurs when the `FundingRateVault` has accrued profits in Synthetix V3 but has not yet settled any orders to convert those theoretical profits into actual sUSD collateral. For instance, imagine that the vault opens a short position in the WETH market and WETH's price falls significantly over a period when there are no user interactions. The vault's position is technically profitable, but that profit remains "unsettled" unless an order is executed or settled in the perps market, causing the position's gains to be reflected as actual sUSD collateral in the `FundingRateVault`'s account. If a user attempts to `redeem` when no other action has occurred for a long time and new PNL has accumulated in the position, the function that checks the vault's SNXUSD collateral (i.e., calling `getCollateralAmount`) might see a balance of zero, even though the vault's position is in the money. Because `_realizeProfit` relies on the vault actually holding sUSD collateral, that user's redemption can fail due to `"InsufficientSynthCollateral"` error. This essentially means the vault's potential PnL remains "dangling" and unclaimable, waiting for an order to settle or for a deposit operation to trigger the normal profit realization flow.

RECOMMENDATION

To mitigate this, a keeper-like mechanism can periodically interact with the vault to force the settlement of any open profit. One way to do this is to submit an essentially empty order or small net adjustment transaction in the same Synthetix market, forcing the perps system to settle any accrued PnL into actual sUSD. The vault then runs its `_realizeProfit` logic, turning the newly realized sUSD into the desired collateral. This keeper should be run every time the `FundingRateVault`'s current PNL in sUSD is higher than a defined threshold.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

RESOLVED

Missing Initialization of ERC20PermitUpgradeable

CATEGORY Logical Error
LOCATION FundingRateVault.sol
REVIEW Remediation Review 1

DESCRIPTION

In the constructor logic, the contract calls `__ERC20_init`, but never calls `__ERC20Permit_init(string memory name)`. Since `ERC20PermitUpgradeable` relies on its own initializer to set up the EIP-2612 domain separator and associated metadata, failing to invoke `__ERC20Permit_init` causes permit functionality to remain uninitialized. This will result in an incorrect domain separator.

RECOMMENDATION

Consider calling `__ERC20Permit_init` in the vault's initializer alongside `__ERC20_init`. For instance:

```
__ERC20_init(_getName(config.marketId), _getSymbol(config.marketId));  
__ERC20Permit_init(_getName(config.marketId));
```

This ensures the contract's domain separator and internal state for permit are fully and securely initialized.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/65>

MEDIUM **M-04**

RESOLVED

Incorrect Fee-on-Fee Calculation In totalAssets Function

CATEGORY Logical Error
LOCATION FundingRateVault: 248–255
REVIEW Remediation Review 1

DESCRIPTION

The `totalAssets()` view function currently calculates potential management and performance fees by calling `_getManagementFeeDebt()` and `_getPerformanceFeeDebt()` with the gross asset value (`_totalAssetsExcludingDebt()`) as input. However, this gross value already implicitly includes the value that will be deducted as currently accrued fee debt (`_managementFeeDebt` and `_performanceFeeDebt`).

This leads to a situation where the potential fees reported by `totalAssets()` are calculated based partly on existing fee debt, effectively resulting in a "fee-on-fee" calculation within this view function. This calculation method is inconsistent with the fee accrual logic in the `_updateManagementFeeDebt()` and `_updatePerformanceFeeDebt()` functions, which correctly calculate new fee increments based on assets net of existing debt before adding the increment to the debt accumulator.

RECOMMENDATION

To ensure consistency and prevent the reporting of slightly inaccurate asset values due to fee-on-fee calculations in the view function, modify the `totalAssets()` function. The calculation should determine the potential fees to be subtracted based on the assets after subtracting the currently accrued fee debt, aligning the view logic with the accrual logic.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/64>

LOW **L-01**

RESOLVED

Overcollected Redemption Fee Due To Price Impact

CATEGORY	Warning
LOCATION	FundingRateVault.sol
REVIEW	Remediation Review 1

DESCRIPTION

During a redemption, the vault calculates the redemption fee as a percentage of the net USDC gained by the vault's contract (`usdcBalance - usdcBefore`). However, this net gain can be reduced by the `_rebalancePosition` call, which may incur slippage and price impact in swapping tokens. In practical terms, the user never "receives" the portion lost to slippage, but the vault still charges a fee on the entire net difference. This can lead to the user effectively paying a redemption fee on amounts they did not truly realize, as the slippage portion never went to them.

RECOMMENDATION

To ensure fairness and clarity, consider adjusting the fee base so it excludes the price-impact portion. Users should pay a redemption fee only on the actual amount received, avoiding a scenario where they are charged on volume lost to slippage.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/63>

LOW **L-02**

RESOLVED

Unnecessary ERC2771 Check

CATEGORY	Warning
LOCATION	FundingRateVault.sol: 269
REVIEW	Remediation Review 1

DESCRIPTION

The `executeOperation` is only intended, and also coded, to be called by the AAVE pool. However, it fetches the caller through the `ERC2771:_msgSender` method. This function is completely useless to be used here because the only caller will be the AAVE pool and it can only call it directly without using the trusted forwarder. Hence, executing this method is used.

RECOMMENDATION

Use the `msg.sender` directly

```
function executeOperation(
    address,
    uint256 amount,
    uint256 premium,
    address initiator,
    bytes calldata params
) external override returns (bool) {
    if (initiator != address(this)) revert NotAuthorized();
    -- if (ERC2771Context._msgSender() != Addresses.AAVE_POOL) {
    ++ if (msg.sender != Addresses.AAVE_POOL) {
        revert NotAuthorized();
    }
    ...
}
```

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/62>

LOW **L-03**

RESOLVED

Misleading Error

CATEGORY	Warning
LOCATION	FundingRateVault.sol: 695
REVIEW	Remediation Review 1

DESCRIPTION

When depositing margin, if there is no amount of `sCollateral` to be added, the `NotEnoughToCoverFees()` error is thrown. This error can be misleading because in this function there is no fee payment and is failing because the amount to deposit is 0. There is an error signature that would fit better for this situation: `ZeroAmount()`.

RECOMMENDATION

Throw the `ZeroAmount` error instead of the `NotEnoughToCoverFees`:

```
function _depositMargin() internal {
    IERC20 collateral = IERC20(_COLLATERAL_ADDRESS);
    uint256 balance = collateral.balanceOf(address(this));
    collateral.approve(Addresses.SPOT_MARKET, balance);
    _spotMarket.wrap(_COLLATERAL_ID, balance, 0);
    IERC20 sCollateral = IERC20(_S_COLLATERAL_ADDRESS);
    balance = sCollateral.balanceOf(address(this));
    -- if (balance == 0) revert NotEnoughToCoverFees();
    ++ if (balance == 0) revert ZeroAmount();
    sCollateral.approve(Addresses.PERPS_MARKET, balance);
    _perpsMarket.modifyCollateral(
        accountId,
        _COLLATERAL_ID,
        int256(balance)
    );
    emit MarginAdded(_S_COLLATERAL_ADDRESS, balance);
}
```

RESOLUTION

Status: Resolved.

L-03 Misleading Error

CONTINUED

<https://github.com/Synthetixio/vaults-contracts/pull/61>

LOW **L-04**

ACKNOWLEDGED

Incorrect Total Assets Cap Check

CATEGORY	Logical Error
LOCATION	FundingRateVault.sol: 149
REVIEW	Remediation Review 1

DESCRIPTION

During deposits the `_validateDeposit` function, it performs some checks before doing any external interaction. One of the checks is to ensure that the `totalAssets()` does not exceed a limit. This check is computed based on `totalAssets()` before updating the management and performance fees and is added to the amount of USDC to deposit. There are 2 things here, the first one is that management and performance fees are updated after executing this check. Thus, if a significant amount of time has passed and positive yield has been accounted, these fees will increase but will not be subtracted from the total assets until they are updated. Hence, the total assets will be greater than it should disabling the user to deposit more funds. The second thing is that the check adds the total assets added with the amount of USDC to deposit to compare it with the assets cap. Using the amount of USDC directly is wrong because the deposit will have to make some swaps and will lose value along the way. Hence, the check should be performed at the end to use the actual amount of assets gained.

RECOMMENDATION

By executing the assets cap at the end ensures that the fees has been updated and the real amount of assets gained plus total assets does not exceed the cap:

```
function deposit(
    uint256 amount,
    address receiver,
    uint256 minShares
) public override returns (uint256 shares) {
    ...
    if (netGain < keeperFee) revert NotEnoughToCoverFees();
    netGain -= keeperFee;
    ++ uint256 newTotalAssets = totalAssets();
    ++ if (newTotalAssets > totalAssetsCap) revert ExceedsTotalAssetsCap();
    shares = _assetsToShares(amount.minimum(netGain), exchangeRateBefore);
    if (shares < minShares) revert NotEnoughShares(shares, minShares);
```

L-04 Incorrect Total Assets Cap Check

CONTINUED

```
_mint(receiver, shares);
emit Deposit(ERC2771Context._msgSender(), receiver, amount, shares);
}

function _validateDeposit(uint256 amount) internal view {
    if (amount == 0) revert ZeroAmount();
    if (paused) revert Paused();
    _validateInsolvency();
    _validateAssetTransactionSize(amount);
    -- uint256 newTotalAssets = totalAssets() + amount;
    -- if (newTotalAssets > totalAssetsCap) revert ExceedsTotalAssetsCap();
}
```

— RESOLUTION

Status: Acknowledged.

LOW **L-05**

RESOLVED

ModifiedPosition event emits wrong referrer

CATEGORY	Logical Error
LOCATION	FundingRateVault.sol: 749
REVIEW	Remediation Review 1

— DESCRIPTION

The `FundingRateVault::_rebalancePosition` function sets the `feesRecipient` as the `referrer` in the `OrderCommitmentRequest` struct. But the `ModifiedPosition` event does not update the `referrer` to the `feesRecipient` address.

— RECOMMENDATION

It is recommended to update the `referrer` address to the `feesRecipient` when emitting the the `ModifiedPosition` event.

— RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/60>

LOW **L-06**

ACKNOWLEDGED

ETH received via receive() can be locked

CATEGORY	Warning
LOCATION	FundingRateVault.sol: 83
REVIEW	Remediation Review 1

— DESCRIPTION

The `FundingRateVault` has implemented the `receive()` function to accept transfer of `ETH` to the contract via `refundETH` execution. The `ETH` can be transferred directly to the contract by mistake as well. But there is no logic in the `FundingRateVault` contract to `unlock` this transferred ETH to the contract.

— RECOMMENDATION

It is recommended to unlock the `received` eth to the `feeRecipient` or to another admin account.

— RESOLUTION

Status: Acknowledged.

HIGH **H-01**

ACKNOWLEDGED

Incorrect Order Slippage Check

CATEGORY Validation
LOCATION FundingRateVault.sol: 696
REVIEW Remediation Review 2

DESCRIPTION

Similar to the original H-01 (Rebalance Extractable Value) issue from the TLX Perps-V3 vaults review, the slippage check for the `_rebalancePosition` function is incorrectly implemented.

The `acceptablePrice` is based upon the `fillPrice` computed by the `perpsMarket`, which includes the price impact that would be experienced by the order in the current transaction. This means there is no protection against a malicious actor frontrunning the rebalance action and skewing the market to force the vault to pay a significant amount of price impact. A malicious actor may be able to extract value this way by back-running the execution of the vault's order to re-correct the skew and receive beneficial impact.

Additionally, if the market is already innocently in a heavily skewed state the vault will automatically accept any amount of price impact that is present at the time of the `_rebalancePosition` call.

RECOMMENDATION

Slippage checks should be based relative to the price of the underlying asset, without factoring in the price impact that would be experienced on the exchange.

Adopt the same logic that is used in the TLX V3 vaults to correctly enforce slippage validations without exposing the vaults to automatic cancellation fees.

References:

H-01 (Rebalance Extractable Value) (main issue that is present now)

H-01 (Vault Drained By Cancellation Fees) (a subtle issue that is introduced if you aren't careful with the remediation!)

RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Losses Perturb Delta Neutrality

CATEGORY	Warning
LOCATION	FundingRateVault.sol
REVIEW	Remediation Review 2

DESCRIPTION

Similarly to M-02 (PnL Realization Depends On Vault Activity) from the remediation review, losses which accrue as debt for the vault can also perturb the delta neutrality of the system and will not be settled with regular deposits.

For example:

- WETH Price is \$1,000 (yes sadly)
- Vault collateral = 1 WETH
- Vault debt is \$500
- availableMargin reports $\$1,000 - \$500 = \$500$ of WETH
- 0.5 WETH exposure is assumed, rebalances target open interest to offset a 0.5 WETH exposure
- The debt is only settled on redemptions so keeper deposit actions will not re-delta-neutral the vault

RECOMMENDATION

Be aware of this behavior and consider consistently paying down any non-trivial debt accrued to the vault to avoid deviation from delta-neutrality.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Slippage in Swaps Can Prevent Flash Loan Repayment

CATEGORY DoS
LOCATION FundingRateVault.sol
REVIEW Remediation Review 2

DESCRIPTION

When the vault calls `realizeFees`, it relies on a flash loan from Aave to repay any Synthetix Perps debt and free up margin. The margin is then swapped twice (collateral → WETH → USDC) to repay the flash loan principal + premium. If the pool liquidity is low or the slippage is large, the final amount of USDC may be insufficient to repay the flash loan. This triggers a revert (ERC20: transfer amount exceeds balance) when repaying the flashloan, blocking the entire fee realization process.

In the Proof of Concept shared, with a fixed 1% slippage:

- The contract only had a 10000 USDC deposit in a whole year.
- The amount that must be repaid is 8421_793776 USDC.
- The vault only has 8280_947269 USDC to repay. Short by 140_846507 USDC.
- Total management fee is 30_246781, way less than all the assets lost due to slippage.

RECOMMENDATION

Consider updating the `realizeFees` function process. I would simplify the whole process and simply take a fixed amount of 1-2% upon a deposit as a management fee. This way we avoid all the losses caused by slippage during the realization of the fees.

PROOF OF CONCEPT <https://gist.github.com/r0bert-ethack/f2ddaf0dafb8e3b4fe7bc645e6e5671a>

RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Typo

CATEGORY	Typo
LOCATION	FundingRateVault.sol: 989
REVIEW	Remediation Review 2

DESCRIPTION

In the `_payRedemptionFee` function the `assetsRedeemed` parameter contains an extra s.

RECOMMENDATION

Rename the parameter to `assetsRedeemed`.

RESOLUTION

Status: Resolved.

<https://github.com/Synthetixio/vaults-contracts/pull/71>

LOW **L-02**

ACKNOWLEDGED

Rebalance Calls Can Fail with ExceedsMarketCreditCapacity

CATEGORY DoS
LOCATION FundingRateVault.sol
REVIEW Remediation Review 2

DESCRIPTION

When the vault calls `_rebalancePosition`, it submits an order in Synthetix V3 that adjusts the short/long position size. However, Synthetix V3 has a per-market credit capacity: if the new order would push the market's total locked credit above the delegated limit, the system reverts with an `ExceedsMarketCreditCapacity` error. This can cause repeated rebalances to fail and create a denial of service scenario for the vault if it always attempts to open or adjust positions in a market that has hit its credit cap. Although this is an edge case, it can happen under certain liquidity crunches or if multiple integrators or large users saturate the market's capacity.

RECOMMENDATION

Document that if Synthetix's credit capacity for this market is exhausted, the vault cannot perform its usual hedging, which may degrade performance or expose depositors to directional risk until capacity returns.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.